

Documentation

E-Commerce

A Women's Fashion E-Commerce Web Application

Team Memembers

Shilpi Kumari (25MCA0038)

Anjana Ann Binoy (25MCA0163)

Aribah Mariyam (25MCA0128)

C.S.Gaayathri (25MCA0193)

Ayushi Kumari (25MCA0177)

a. Software Requirement

1.1 Introduction

The **Microservices-Based E-commerce System** is a web application designed to facilitate online buying and selling of products. The system allows users to register, browse products, place orders, communicate with sellers, and receive notifications.

Unlike traditional monolithic systems, this project follows a **microservices architecture**, where each service is independent and handles a specific functionality. This improves scalability, maintainability, and flexibility.

The system is developed using:

- React / Typescript for frontend
- Node.js with Express.js for backend
- MongoDB for data storage

1.2 Functional Requirements

User Management

- User registration and login
- Role-based access (Buyer, Seller, Admin)
- Authentication and authorization

Product Management

- Sellers can add, update, delete products
- Buyers can browse and search products

Order Management

- Buyers can place orders
- Order tracking functionality
- Order status updates (Pending, Shipped, Delivered)

Inventory Management

- Maintain product stock levels
- Automatic stock updates after orders

Notification System

- Notify sellers when orders are placed
- Notify users about order updates

Chat System

- Real-time messaging between buyer and seller
- Allow buyers to ask product-related questions
- Help sellers respond quickly to customers

1.3 Non-Functional Requirements

- **Scalability:** Each service can scale independently
- **Performance:** Faster response due to distributed services
- **Security:** JWT-based authentication
- **Availability:** Services run independently on different ports
- **Maintainability:** Easy to update or modify individual services
- **Reliability:** Failure of one service does not affect others

1.4 Assumptions and Constraints

Assumptions

- Users have internet access
- Each service runs independently

- MongoDB is available for all services

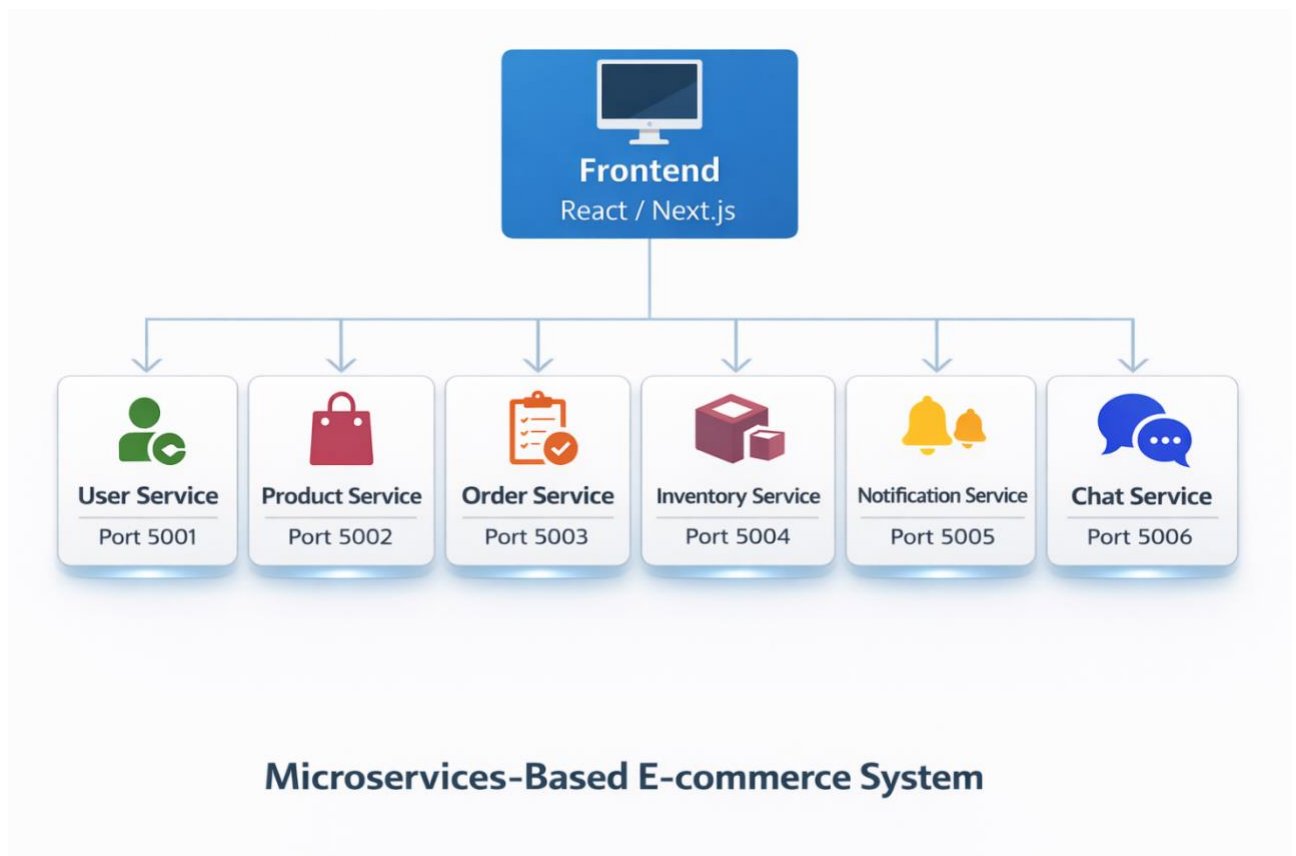
Constraints

- No API Gateway used
- Services communicate via REST APIs only
- Limited to basic real-time chat (no WebSockets)

b. Layout Design / System Architecture

2.1 Architecture Explanation

- The system follows a **Microservices Architecture** where each service is independently developed and deployed. Each service runs on a separate port and communicates using REST APIs.
- There is no API Gateway, so the frontend directly interacts with individual services using Axios.



2.2 Technology Stack

- **Frontend:** React / Next.js
- **Backend:** Node.js + Express
- **Database:** MongoDB

- **API Communication:** REST (Axios)
- **Authentication:** JWT

Step-by-Step Architectural Flow of the System

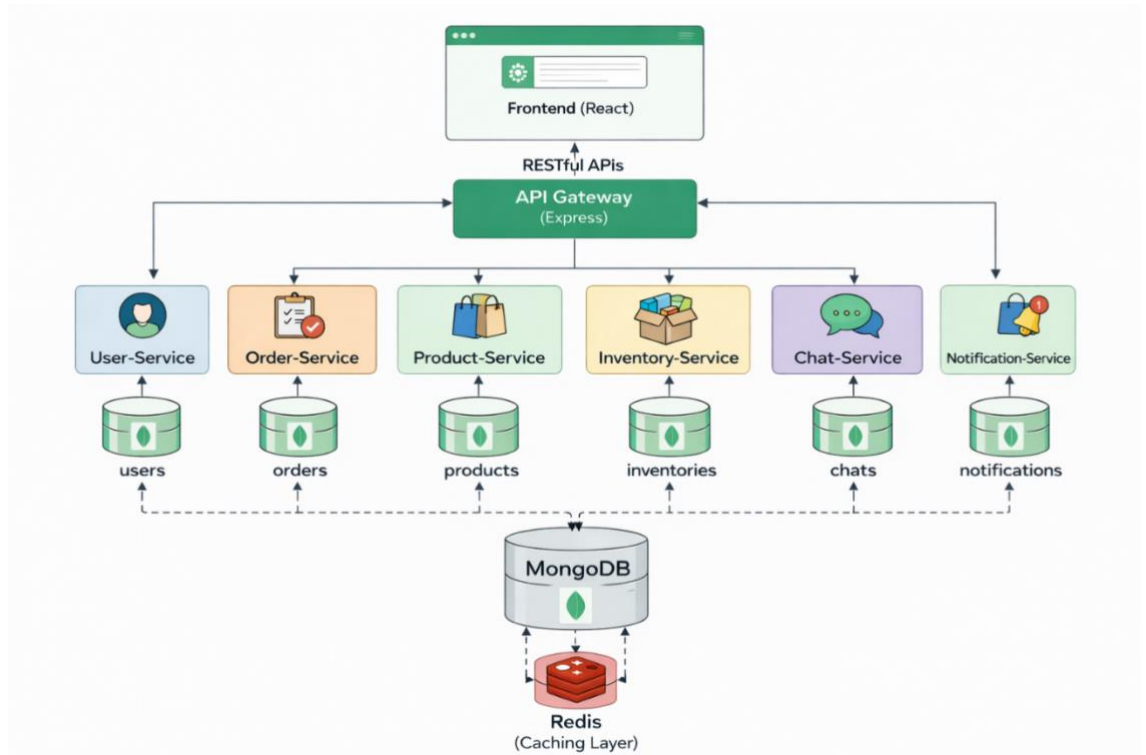


The flow starts from the frontend, where the user logs in using the User Service. Then products are fetched from the Product Service. When the user places an order, the Order Service processes it, updates inventory, and triggers notifications. Chat Service is used for communication.

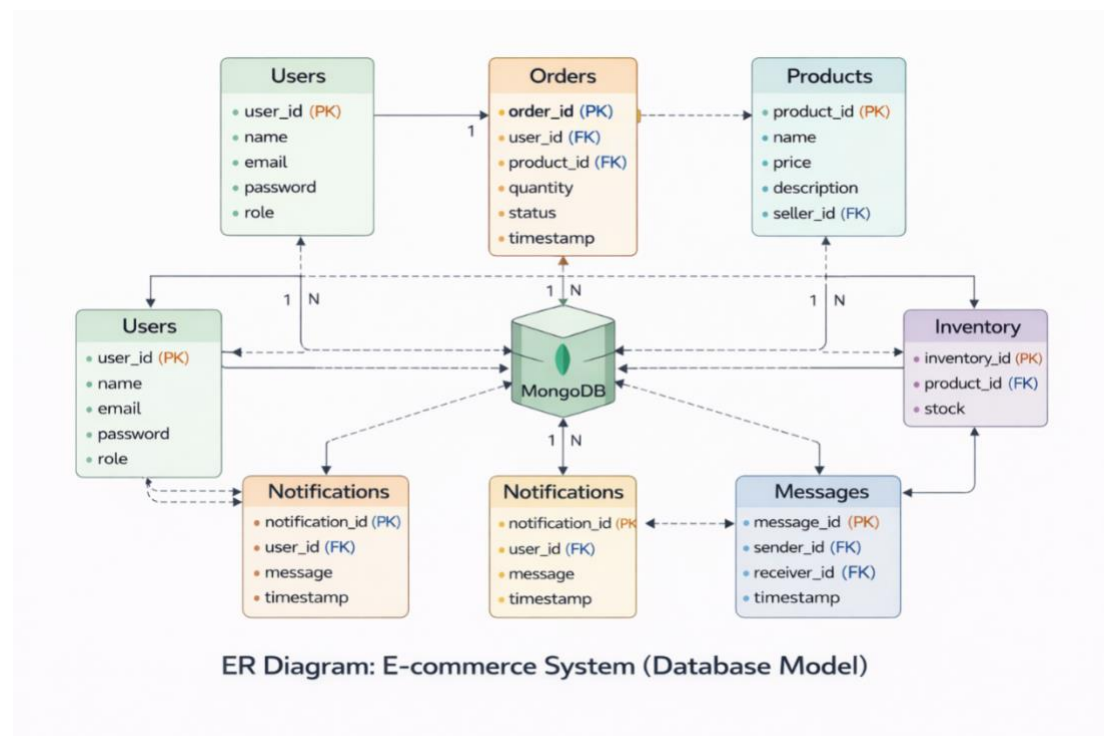
- **User Service** → Handles authentication and roles
- **Product Service** → Manages product data
- **Order Service** → Processes orders
- **Inventory Service** → Manages stock
- **Notification Service** → Sends alerts
- **Chat Service** → Enables communication

Database Design

The system follows a **database-per-service approach**, where each microservice manages its own data independently using MongoDB. This ensures loose coupling, better scalability, and easier maintenance of the system.



Each microservice maintains its own database collection. Data flows from users to products, then to orders, which updates inventory and triggers notifications, while chat messages are stored separately. Additionally, **Redis** is used as a caching layer to store frequently accessed data, improving system performance and reducing database load. This ensures modular, scalable, and efficient data handling.



The ER diagram shows how users, products, orders, inventory, notifications, and messages are related using primary and foreign keys to manage data efficiently.

C. Source Code Description

GITHUB Link - <https://github.com/Ayushii-Sharma20/E-commerce>

Backend Structure

Each microservice contains:

- Controllers
 - Routes
 - Models
 - Services
-

Microservices Structure

- user-service
- product-service
- order-service
- inventory-service
- notification-service

JWT implemetation

```
services > chat-service > middleware > JS authMiddleware.js > ...
1  const jwt = require("jsonwebtoken");
2
3  module.exports = (req, res, next) => {
4    const authHeader = req.header("Authorization") || req.header("authorization");
5
6    if (!authHeader || !authHeader.startsWith("Bearer ")) {
7      return res.status(401).json({
8        success: false,
9        message: "Authorization token is required",
10       error: "Missing bearer token"
11     });
12   }
13
14   try {
15     const token = authHeader.split(" ")[1];
16     const decoded = jwt.verify(token, process.env.JWT_SECRET);
17     const userId = decoded.userId || decoded.id;
18
19     if (!userId || !decoded.role) {
20       return res.status(401).json({
21         success: false,
22         message: "Invalid token payload",
23         error: "Token must include userId and role"
24       });
25     }
26   }
```

Inventory

```
const Inventory = require("../models/Inventory");

const sendError = (res, statusCode, message, error = null) =>
  res.status(statusCode).json({
    success: false,
    message,
    error: error ? (typeof error === "string" ? error : error.message) : null
  });

const sendSuccess = (res, statusCode, message, data = {}) =>
  res.status(statusCode).json({
    success: true,
    message,
    ...data
  });
```

DataBase Connectivity

```
services / inventory-service / db.js / ...
1   const mongoose = require("mongoose");
2
3   const connectDB = async () => {
4     try {
5       await mongoose.connect(process.env.MONGO_URI);
6       console.log("MongoDB Connected ✅");
7     } catch (err) {
8       console.error(err.message);
9       process.exit(1);
10    }
11  };
12
13  module.exports = connectDB;
```

RabbitMQ implementation

```

1  const amqp = require('amqplib')
2
3  let channel
4
5  async function connectQueue() {
6      const connection = await amqp.connect('amqp://127.0.0.1')
7      channel = await connection.createChannel()
8
9      await channel.assertQueue('notifications')
10
11     console.log("RabbitMQ Connected (Notification Service)")
12 }
13
14 function getChannel() {
15     return channel
16 }
17
18 module.exports = { connectQueue, getChannel }

```

Role based access

```

1  module.exports = (...inputRoles) => {
2      const roles =
3          inputRoles.length === 1 && Array.isArray(inputRoles[0]) ? inputRoles[0] : inputRoles;
4
5      return (req, res, next) => {
6          if (!req.user) {
7              return res.status(401).json({
8                  success: false,
9                  message: "Unauthorized access",
10                 error: "Authentication required"
11             });
12         }
13
14         if (!roles.includes(req.user.role)) {
15             return res.status(403).json({
16                 success: false,
17                 message: "Access denied",
18                 error: `Required role: ${roles.join(", ")}`
19             });
20         }
21
22         return next();
23     };
24 };
25

```

User service

```

const connectDB = require("../db");
const userRoutes = require("../routes/userRoutes");

const app = express();

app.use(cors());
app.use(express.json());

connectDB();

app.use("/api/users", userRoutes);

app.listen(process.env.PORT, () => {
  console.log(`User Service running on port ${process.env.PORT}`);
});

```

API Communication

Services communicate using REST APIs via Axios.

```

services > inventory-service > JS index.js > ...
1  require("dotenv").config();
2  const express = require("express");
3  const cors = require("cors");
4  const connectDB = require("../db");
5
6  const app = express();
7
8
9  app.use(cors());
10 app.use(express.json());
11
12
13 connectDB();
14
15 // routes
16 app.use("/api/inventory", require("../routes/inventoryRoutes"));
17
18
19 const PORT = process.env.PORT || 3004;
20
21 app.listen(PORT, () => {
22   console.log(`Inventory Service running on port ${PORT}`);
23 });
24 console.log(require("../routes/inventoryRoutes"));

```

D. Execution Screenshots

- Login/Register page

StyleSphere

Create Account

Join StyleSphere today

Name
Ayushi Sharma

Email
ayushi20@gmail.com

Password

Register As

Already have an account? [Login](#)

localhost:3000 says
Registered ■

Create Account

Join StyleSphere today

Name
Ayushi Sharma

Email
ayushi0@gmail.com

Password

Register As

Already have an account? [Login](#)

StyleSphere

Welcome Back

Enter your credentials to access your account

Email
jane@example.com

Password [Forgot password?](#)
Enter your password

Login As

This only changes the redirect target. Your real access is based on your account role.

Don't have an account? [Create one](#)

[← Back to home](#)

The login and registration module allows users to create an account and access the system as a Buyer, Seller, or Admin, ensuring role-based authentication and access control.

localhost:3000/profile

StyleSphere

Home Shop Orders Profile

My Profile

Manage your account settings and view order history

Personal Information

Update your personal details



Shilpi jaiswal
Member since January 2024

Full Name
Shilpi jaiswal

Phone Number
Not provided

Email Address
shilpijaiswal02@gmail.com

Quick Actions

Recent Orders

No orders yet

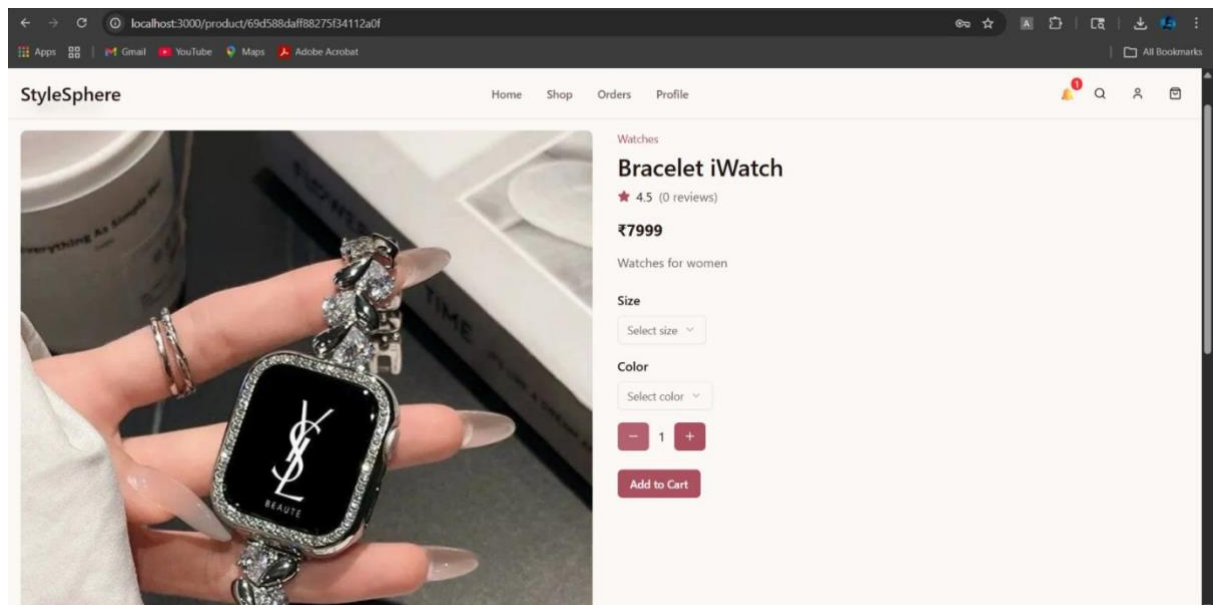
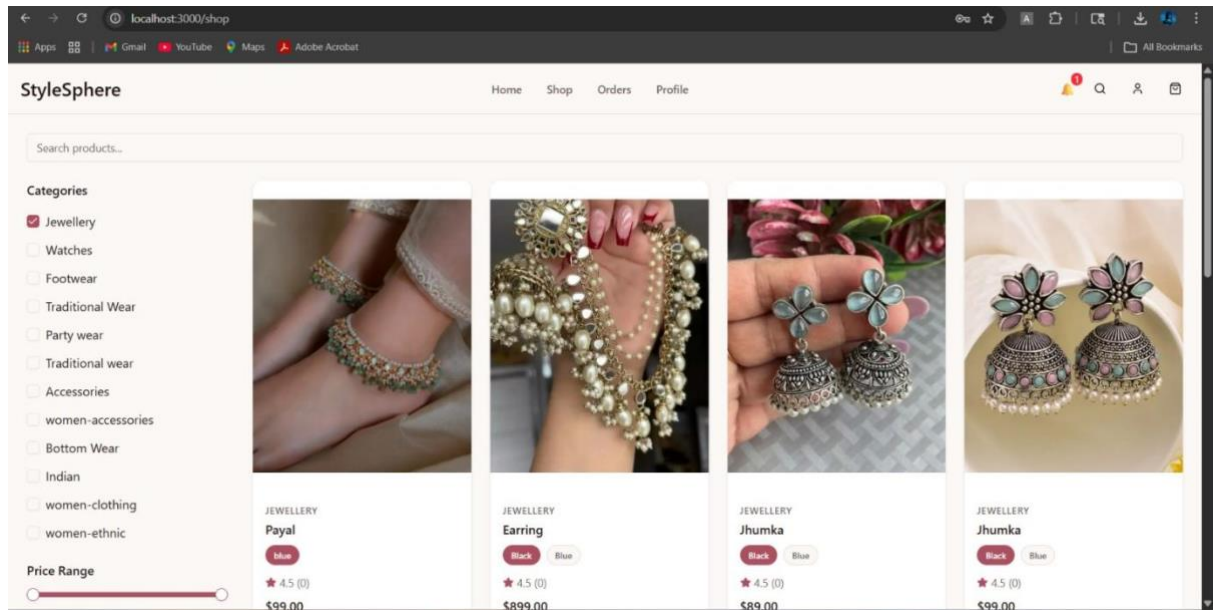
Saved Addresses

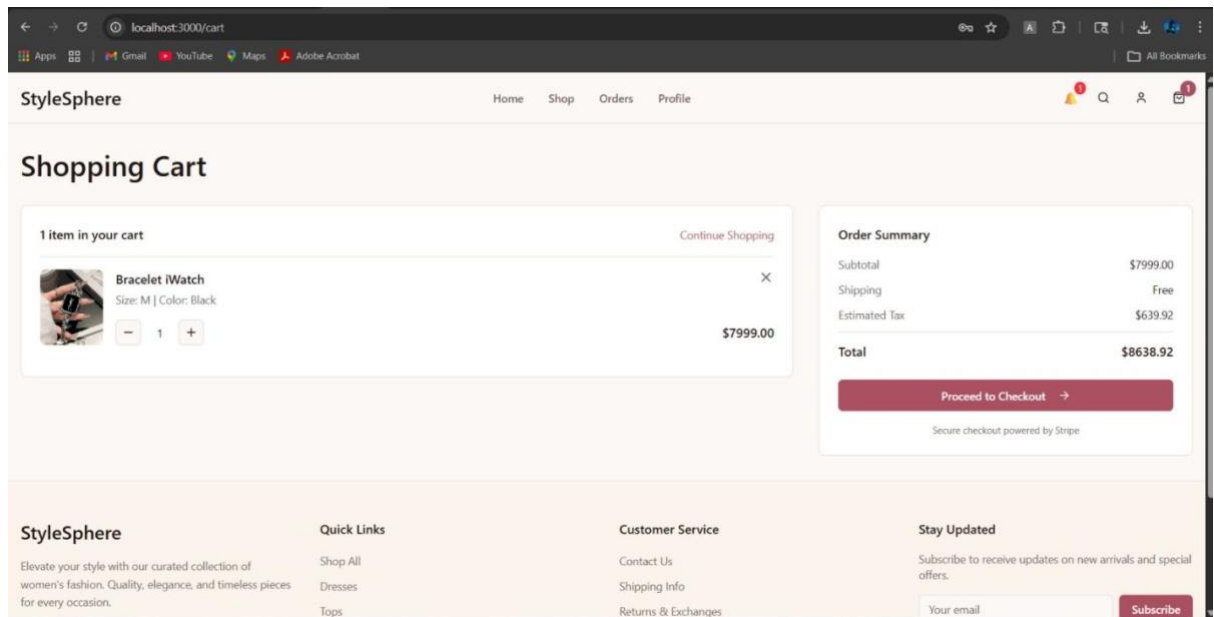
Manage your shipping addresses

No addresses yet

- **Product Listing (Buyer View)**

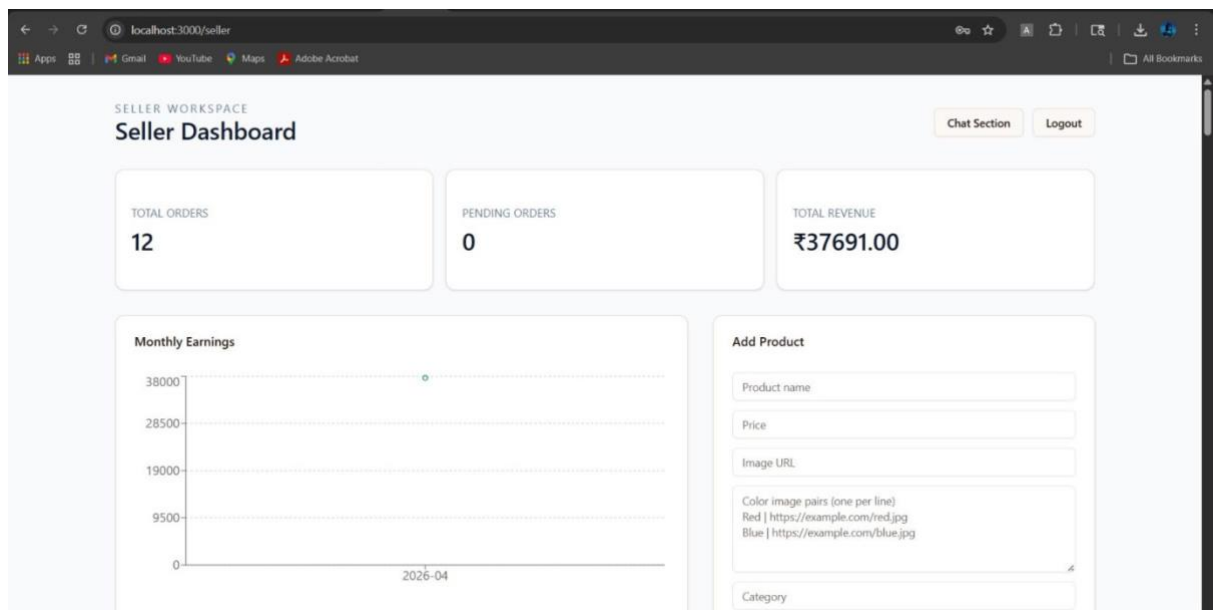
Buyers can browse and view available products with details such as price, description, and seller information.





- **Product Creation (Seller Panel)**

Sellers can add new products by providing product details such as name, price, description, and stock information.



Your Orders		
Order #ee0cb	Revenue: ₹7999.00	SHIPPED
Bracelet iWatch		1 x ₹7999
Mark Delivered		
Order #8872bd	Revenue: ₹2500.00	CONFIRMED
Sharara Set		1 x ₹2500
Ship		
Order #6de0b8	Revenue: ₹999.00	CONFIRMED

Your Products		
 JEWELLERY Payal ₹99 Status: APPROVED Colors: blue	 WATCHES Bracelet iWatch ₹7999 Status: APPROVED Colors: Black, White	 FOOTWEAR Heels ₹999 Status: APPROVED Colors: Red

- **Order Placement (Buyer)**

Buyers can place orders for selected products, which are processed by the Order Service and stored in the system.

← → ↻ 🌐 localhost:3000/checkout

📱 🌐 📧 Gmail 📺 YouTube 📍 Maps 📄 Adobe Acrobat

🔍 📌 All Bookmarks

StyleSphere

HomeShopOrdersProfile

🔔 🔍 👤 📧

[← Back to Cart](#)

Checkout

📦 Shipping Information

Full Name

Anjana Ann Binoy

Email

anjanna123@gmail.com

Phone

9988979797

Address

Vit university

City

vellore

State

Tamil Nadu

ZIP Code

643004

Order Summary

 Bracelet iWatch
M / Black x1

\$7999.00

Subtotal

\$7999.00

Shipping

Free

Tax

\$639.92

Total

\$8638.92

Place Order - \$8638.92

← → ↻ 🌐 localhost:3000/order-confirmation?id=69daade9b4f4c58f71eef0cb

📱 🌐 📧 Gmail 📺 YouTube 📍 Maps 📄 Adobe Acrobat

🔍 📌 All Bookmarks

StyleSphere

HomeShopOrdersProfile

🔔 🔍 👤 📧

🟢

Order Confirmed!

Thank you for your purchase. Your order has been received.

Order Number
69daade9b4f4c58f71eef0cb

📦 Estimated Delivery

3-5 business days

A confirmation email has been sent with order details.

View Order Details

Continue Shopping →

← → ↻ 🌐 localhost:3000/orders/69daade9b4f4c58f71eef0cb

📱 🌐 📧 Gmail 📺 YouTube 📍 Maps 📄 Adobe Acrobat

🔍 📌 All Bookmarks

StyleSphere

HomeShopOrdersProfile

🔔 🔍 👤 📧

Order Details

Order ID
69daade9b4f4c58f71eef0cb

Placed on 4/12/2026

Total: ₹7999

Items

 Bracelet iWatch
Quantity: 1
Price: ₹7999

₹7999

Notifications

Order 69daade9b4f4c58f71eef0cb placed successfully

Order 69d713c2d47c6618756de0b7 placed successfully

CONFIRMED

StyleSphere

Elevate your style with our curated collection of

Quick Links

Shop All

Customer Service

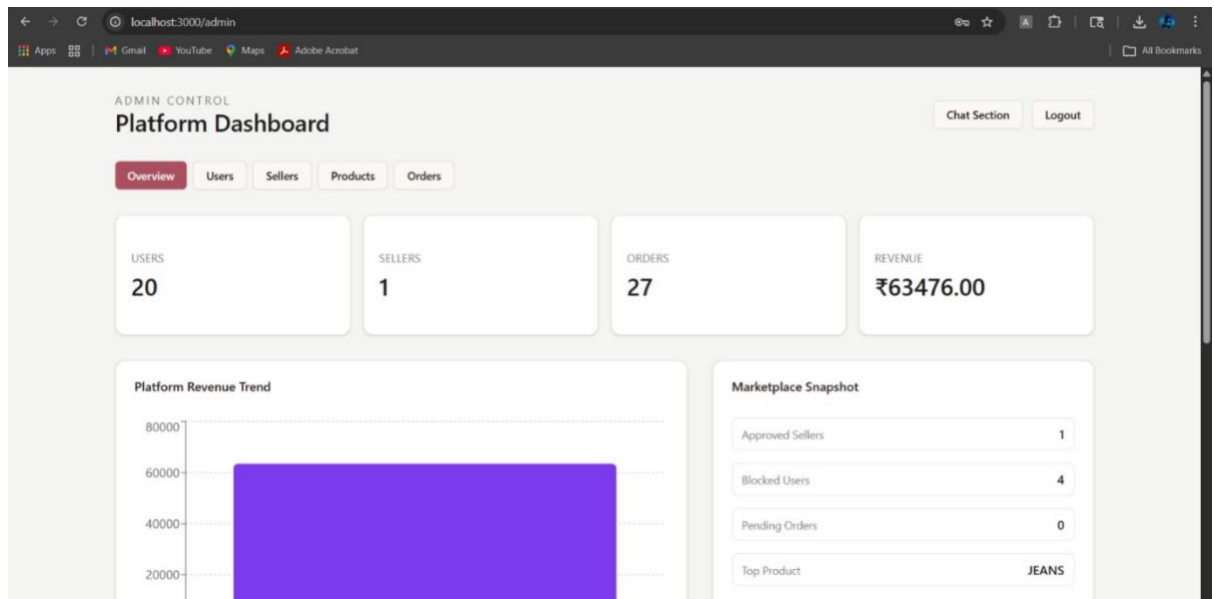
Contact Us

Stay Updated

Subscribe to receive updates on new arrivals and special offers.

- **Admin Dashboard**

The admin panel provides system monitoring features, allowing the admin to view users, products, orders, and overall system activity.



This screenshot shows the 'Users' tab of the Admin Dashboard. The page features a top navigation bar with 'Chat Section' and 'Logout' buttons. Below the navigation bar, there are five tabs: 'Overview', 'Users' (selected), 'Sellers', 'Products', and 'Orders'. The main content area displays a list of three users in white boxes. Each user entry includes their name, email address, role, and a button to manage their status (Block or Unblock). The browser's address bar shows 'localhost:3000/admin'.

Name	Email	Role	Action
Ayushi Sharma	ayushi0@gmail.com	Buyer	Block
ayushi	ayushi02@gmail.com	Buyer	Block
ayushi	ayushi20@gmail.com	Buyer	Unblock

ADMIN CONTROL

Platform Dashboard

Chat SectionLogout

OverviewUsersSellersProductsOrders

sellerTest

seller@gmail.com

Status: Approved

ApproveReject

Admin Messaging Hub

Handle buyer support and seller coordination directly from the admin dashboard.

No contacts available yet.

Choose a buyer or seller conversation from the admin messaging hub.

ADMIN CONTROL

Platform Dashboard

Chat SectionLogout

OverviewUsersSellersProductsOrders



JEWELLERY

Payal

Seller: 69d408a66f1db023591c23ed

Status: APPROVED



WATCHES

Bracelet iWatch

Seller: 69d408a66f1db023591c23ed

Status: APPROVED



FOOTWEAR

Heels

Seller: 69d408a66f1db023591c23ed

Status: APPROVED

ADMIN CONTROL

Platform Dashboard

Chat SectionLogout

OverviewUsersSellersProductsOrders

Order #eef0cb

DELIVERED

Buyer: 69d53fab906841fb10a69900

Items: 1

₹7999.00

Order #88728d

CONFIRMED

Buyer: 69d40e606f1db023591c23f1

Items: 1

₹2500.00

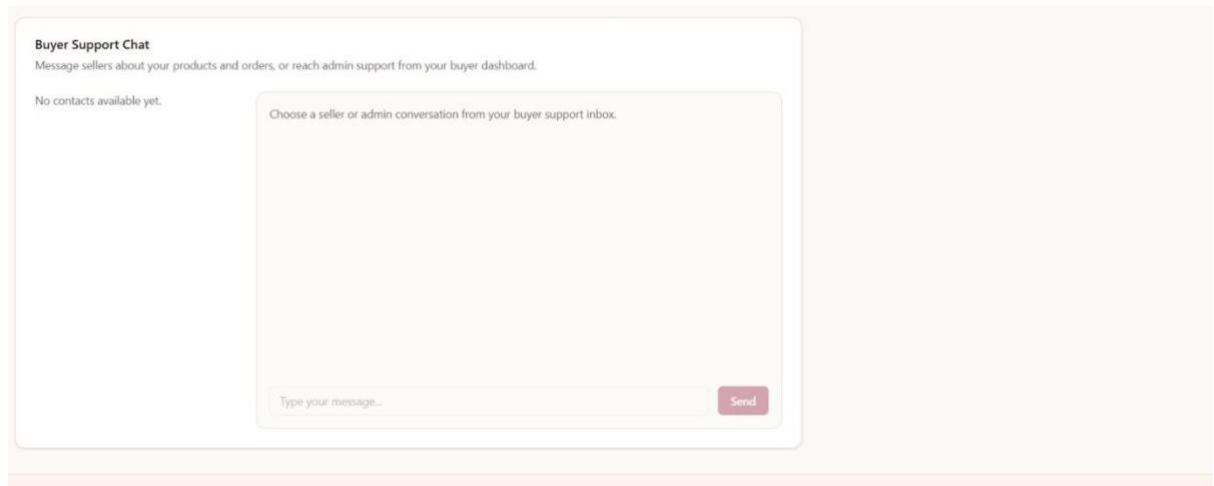
Order #6de0b8

CONFIRMED

The admin logs in through the User Service and accesses the dashboard, where data from user, product, order, inventory, and notification services is displayed for monitoring and management.

- **Chat Interface (Buyer–Seller)**

The chat system allows real-time communication between buyers and sellers for product inquiries and order-related discussions.



Running Services (Backend)

All microservices run independently on different ports, demonstrating the distributed microservices architecture.

```
node - frontend + - - - - - x
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\shilp\Downloads\E-commerce-main\frontend> npm run dev
✓ Compiled /shop in 911ms (1800 modules)
GET / 200 in 350ms
GET / 200 in 180ms
GET / 200 in 172ms
GET /login 200 in 183ms
o Compiling /seller ...
✓ Compiled /seller in 4.2s (2042 modules)
o Compiling /admin ...
✓ Compiled /admin in 1371ms (2052 modules)
o Compiling /product/[id] ...
✓ Compiled /orders in 7.4s (2124 modules)
o Compiling /cart ...
✓ Compiled /cart in 2.2s (2098 modules)
o Compiling /checkout ...
✓ Compiled /checkout in 2s (2118 modules)
o Compiling /order-confirmation ...
✓ Compiled /order-confirmation in 2s (2084 modules)
o Compiling /orders ...
✓ Compiled /orders in 630ms (1056 modules)
o Compiling /orders/[id] ...
✓ Compiled /orders/[id] in 1927ms (2091 modules)
o Compiling /profile ...
✓ Compiled /profile in 3.3s (2108 modules)
GET /seller 200 in 1150ms
GET /admin 200 in 1302ms
GET /admin 200 in 194ms

```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\shilp\Downloads\E-commerce-main\services\user-service> npm run dev
PS C:\Users\shilp\Downloads\E-commerce-main\services\user-service> npm run dev

> user-service@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node index.js'
[dotenv@17.3.1] injecting env (3) from .env -- tip: specify custom .env file path with { path: '/custom/path/.env' }
User Service running on port 3001
MongoDB Connected
```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shilp\Downloads\E-commerce-main\services\order-service> npm run dev

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
◇ injecting env (3) from .env // tip: ~ auth for agents [www.vestauth.com]
MongoDB Connected (Order Service)
🐇 RabbitMQ Connected (Order Service)
🚀 Order Service running on port 3003
📦 Event Sent: {
  userId: '69d53fab906841fb10a69900',
  message: 'Order 69daade9b4f4c58f71eef0cb placed successfully',
  type: 'order'
}

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\shilp\Downloads\E-commerce-main\services\product-service> npm run dev
> product-service@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
◇ injected env (3) from .env // tip: ✎ enable debugging { debug: true }
✎ Product Service running on port 3002
✎ Connecting to Redis...
✔ Redis connected
✔ MongoDB Connected (Product Service)
✔ MongoDB fetch: products:approved
✔ Redis cache set: products:approved
✔ Redis cache hit: products:approved
✔ MongoDB fetch: products:seller:69d408a66f1db023591c23ed
✔ Redis cache set: products:seller:69d408a66f1db023591c23ed
✔ Redis cache hit: products:seller:69d408a66f1db023591c23ed
✔ MongoDB fetch: products:all
✔ Redis cache set: products:all
✔ Redis cache hit: products:all
✔ Redis cache hit: products:all
✔ MongoDB fetch: products:approved
✔ Redis cache set: products:approved
✔ Redis cache hit: products:approved
✔ MongoDB fetch: products:approved
✔ Redis cache set: products:approved
✔ Redis cache hit: products:approved

```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
node - inventory-service + - - - - -

PS C:\Users\shilp\Downloads\E-commerce-main\services\inventory-service> npm run dev

Layer {
  handle: [Function: handle],
  keys: [],
  name: 'handle',
  params: undefined,
  path: undefined,
  slash: false,
  matchers: [Array],
  route: [Route]
},
Layer {
  handle: [Function: handle],
  keys: [],
  name: 'handle',
  params: undefined,
  path: undefined,
  slash: false,
  matchers: [Array],
  route: [Route]
}
}
}
Inventory Service running on port 3004
MongoDB Connected
(node:2832) [MONGODBSE] Warning: mongoose: the 'new' option for 'findOneAndUpdate()' and 'findOneAndReplace()' is deprecated. Use 'returnDocument: 'after'' instead.
(Use 'node --trace-warnings ...' to show where the warning was created)
(node:2832) [MONGODBSE] Warning: mongoose: the 'new' option for 'findOneAndUpdate()' and 'findOneAndReplace()' is deprecated. Use 'returnDocument: 'after'' instead.
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
node - notification-service + - - - - -

PS C:\Users\shilp\Downloads\E-commerce-main\services\notification-service> npm run dev
PS C:\Users\shilp\Downloads\E-commerce-main\services\notification-service> npm run dev

> notification-service@1.0.0 dev
> nodemon app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node app.js'
◇ injecting env (3) from .env // tip: multiple files { path: ['.env.local', '.env'] }
Notification DB connected: ac-bqkdsaw-shard-00-00.hnctmib.mongodb.net
RabbitMQ Connected (Notification Service)
Listening to queue...
Notification Service running on port 3005
Received Event: {
  userId: '69d53fab906841fb10a69900',
  message: 'Order 69daade9b4f4c58f71eef8cb placed successfully',
  type: 'order'
}
Notification saved: new ObjectId('69daadea5a9b5393801e18b2')
```

e. Conclusion

The Microservices-Based E-commerce System successfully demonstrates the implementation of a scalable and modular web application using the MERN stack. The system is designed with independent microservices such as User, Product, Order, Inventory, Notification, and Chat services, each handling a specific functionality.

By using a microservices architecture, the system achieves better scalability, flexibility, and fault isolation compared to traditional monolithic systems. The application supports essential e-commerce features including user authentication, product management, order processing, inventory tracking, notifications, and buyer-seller communication.

Additionally, the use of REST APIs for communication and Redis for caching enhances system performance and responsiveness. Overall, the project provides a practical understanding of modern system design and real-world application development.

Future Work

The system can be further enhanced with additional features and improvements to make it more robust and production-ready:

- Integration of an **API Gateway** for centralized request handling
- Implementation of **real-time chat using WebSockets (Socket.IO)**
- Integration of **payment gateway** for online transactions
- Addition of **recommendation system** based on user behavior
- Offers during special occasions